

Master Roller Shutter Control V84

Home Assistant Blueprint – Sun-Position Shading, Extreme-Heat Protection, Ventilation and Walkthrough Logic

Full documentation · As of July 2026 · Blueprint domain: `automation`

Contents

1. What the blueprint does
2. How it works: the priority chain
3. Prerequisites and required entities
4. Creating helpers and template sensors
5. Window sensors: tilt vs. fully open
6. Installing the blueprint
7. Complete input reference
8. Setting recommendations by orientation
9. The manual override
10. Interaction with a fire-alarm automation (incl. acknowledged reset and dashboard card)
11. Troubleshooting and debugging
12. Known limitations
13. Changelog

1. What the blueprint does

The blueprint controls **a single roller shutter** automatically throughout the day. One automation is created per shutter, so every room can have its own facade orientation, temperature thresholds and positions.

It combines six jobs that would otherwise require separate automations:

- **Morning opening** – based on time, sunrise, presence, weekday, public holiday and house mode.
- **Sun-position shading** – the shutter only lowers when the sun actually hits *this* facade (azimuth window per orientation), it is bright enough (PV power or lux) *and* the temperature justifies it.
- **Extreme-heat protection** – at very high indoor *and* outdoor temperature the shutter moves to a separate, further-closed position. With hysteresis to prevent flapping.
- **Ventilation position** – a tilted window keeps the shutter at a defined minimum height.
- **Walkthrough and cross-ventilation logic** – a fully opened window/patio door raises the shutter so you are neither locked in nor forced to intervene manually.
- **Evening closing** – after sunset with an adjustable offset, taking open windows into account.

Core idea: the blueprint never decides on a single condition, but on a fixed *priority chain*. Safety beats comfort, comfort beats energy.

2. How it works: the priority chain

On every trigger the blueprint runs through a `choose` chain from top to bottom. The **first** matching branch wins; all others are skipped.

Prio	Branch	Condition and effect
0	Fire alarm	If the fire-alarm boolean is <code>on</code> , execution stops immediately with <code>stop</code> . The blueprint moves <i>nothing</i> – it holds still so it does not counteract an external fire-alarm automation (which opens all shutters as an escape route). See chapter 10.
1a	Walkthrough / cross-ventilation	A sensor in <i>window fully open</i> reports <code>on</code> . The shutter raises: • if <i>no</i> shading is active → to <code>max_oeffnung</code> (fully up) • if shading or extreme-heat protection is active → only to <code>durchgangs_pos</code> Works at night, before morning opening time, and out of a manually closed state . The shutter is never <i>lowered</i> here, only raised.
1b	Ventilation (tilt)	A sensor in <i>window ventilation</i> reports <code>on</code> and no fully-open sensor is active. The shutter raises at least to <code>lueftungs_pos</code> so the slat curtain does not rest on the tilted sash.
2	Evening check	At the configured control time (default 22:00) the blueprint checks whether a window is still fully open and sends a push notification.
3	Closing	Night (sun below horizon) and no window fully open → shutter closes. With a tilted window it closes only down to <code>lueftungs_pos</code> . On the sunset trigger the configured <code>sunset_offset</code> is awaited first.
4a	Extreme-heat protection	Sun on the facade <i>and</i> indoor temperature above the ON threshold <i>and</i> outdoor temperature above the ON threshold → shutter moves to <code>hitze_extrem_pos</code> . The mode ends only once indoor <i>or</i> outdoor temperature falls below its respective OFF threshold (hysteresis).
4b	Normal shading	Sun on the facade <i>and</i> heat alarm <i>and</i> enough light (PV or lux) → shutter moves to the shading position. Configurable separately for morning and afternoon.
4c	Raising	No sun protection needed (sun gone, cloudy, cool) → shutter moves to <code>max_oeffnung</code> .

When does the automation run at all?

Triggers are: state changes of the window sensors, presence sensors, PV power (with 2 min debounce), sunset, the *latest opening* and *evening check* times, a change of house mode – plus a `time_pattern` every 15 minutes as a safety net. The mode is `restart` : a new trigger aborts a running run.

The three sun conditions in detail

- `sonne_auf_f` – the sun's azimuth lies within the angle range of the chosen orientation *and* the elevation is above 5°. Prevents shading when the sun is not on the window at all.
- `sonne_stark` – PV power above threshold or lux above threshold, each cross-checked against the weather entity. If a sensor fails, a failover works via the weather alone.
- `hitze_alarm` – linked with **OR**: indoor temperature above the indoor threshold, *or* a rising temperature trend, *or* outdoor temperature above the outdoor threshold. A winter lock (`raumtemp_min_schwelle`) suppresses everything while the room is too cool.

Important to understand: because `hitze_alarm` is an OR link, a very low outdoor threshold (e.g. 10 °C) makes the indoor temperature practically irrelevant – above 10 °C outside the condition is always true in summer, and only sun position and brightness decide. This can be intentional (»the sun blinds me even at 15 °C«) but should be a deliberate choice.

3. Prerequisites and required entities

What	Required	Description
<code>cover.*</code>	yes	Shutter with position support (<code>set_cover_position</code> and attribute <code>current_position</code>). Pure open/close drives do not work.
<code>sun.sun</code>	yes	Standard integration. Provides azimuth and elevation.
<code>weather.*</code>	yes	Any weather entity. Used for cross-checking and as a failover.
Indoor temperature sensor	yes	<code>sensor</code> with <code>device_class: temperature</code> .
Outdoor temperature sensor	yes	Own sensor or e.g. OpenWeatherMap.
<code>input_select</code> house mode	yes	Helper with at least the options <code>Zuhause</code> , <code>Abwesend</code> , <code>Urlaub</code> .
<code>input_boolean</code> fire alarm	yes	Set by the external fire-alarm automation.
Window sensor tilt	no	<code>binary_sensor</code> , multiple selection possible.
Window sensor fully open	no	<code>binary_sensor</code> , multiple selection possible. Leave empty for windows that are never fully opened.
Presence sensor	no	Multiple selection. Triggers the morning »go signal«.
PV power sensor	no	W or kW – the unit is detected and converted automatically.
Illuminance sensor	no	<code>device_class: illuminance</code> . One sensor per facade side is ideal.
Trend sensor	no	<code>binary_sensor</code> , <code>on</code> = room temperature rising. See chapter 4.
Workday sensor	no	For holiday detection. See chapter 4.
<code>timer</code> notification pause	no	Suppresses push notifications for a period.
Notify service	no	Recommended: a notify group, see chapter 4.

4. Creating helpers and template sensors

4.1 House mode (input_select)

Settings → Devices & Services → Helpers → Create Helper → *Dropdown*. Name: `Hausmodus` . Write the options exactly like this – the blueprint compares text values:

```
Zuhause
Abwesend
Urlaub
```

Effect: `Urlaub` switches to the late opening time. `Abwesend` and `Urlaub` allow opening even without a presence signal.

4.2 Fire-alarm status (input_boolean)

Create Helper → *Toggle*. Name e.g. `Brandalarm Status` → entity `input_boolean.brandalarm_status` . This boolean is the interface between the smoke-detector automation and the shutter control.

4.3 Notification pause (timer)

Create Helper → *Timer*. Name e.g. `Rollo Pause für Benachrichtigung` . While this timer runs, the push notification is treated as suppressed.

4.4 Workday sensor (holidays)

Settings → Devices & Services → Add Integration → **Workday**. Choose country and region, mark the weekend as non-working days. Result: `binary_sensor.workday_sensor` , which is `off` on weekends *and* public holidays.

The blueprint uses it as follows: `off` → the late opening time applies (weekend/holiday). If the input is left empty, the blueprint falls back to a plain weekday check – holidays are then **not** detected.

4.5 Trend sensor for room temperature

The trend sensor makes shading anticipatory: if the room temperature is rising, shading kicks in *before* the threshold is reached. Create it via the **Trend** integration (Settings → Devices & Services → Add Integration → Trend) or via YAML:

```
binary_sensor:
- platform: trend
  sensors:
    temp_wohnzimmer_rollo:
```

```
friendly_name: "Temp living room rising"
entity_id: sensor.temperatur_wohnzimmer
sample_duration: 1800      # 30-minute observation window
min_gradient: 0.0005       # approx. +0.9 K/h
device_class: heat
```

Create one such sensor per room and enter it in the respective automation.

4.6 Notify group (recommended)

Instead of entering the device service in every automation, a notify group should be used. When changing devices you then only change *one* line. In `configuration.yaml`:

```
notify:
  - platform: group
    name: handys
    services:
      - service: mobile_app_my_phone
      - service: mobile_app_partner_phone    # optional
```

Then **restart** Home Assistant (a reload is not enough for notify platforms). In the blueprint you then enter only `handys` as the *phone notification service* – the blueprint builds `notify.handys` from it.

Notify groups are legacy services and create **no entity**. They therefore do not appear under »States«, only under Developer Tools → *Actions*.

5. Window sensors: tilt vs. fully open

The blueprint distinguishes two window states because they have different consequences:

- **Tilted** → the shutter may lower, but only down to the ventilation position (otherwise the curtain rests on the sash).
- **Fully open** → the shutter must raise (walkthrough, cross-ventilation, no lock-in).

A single reed contact **cannot** tell these apart – it only sees »sash moving away from the frame«. There are three practical approaches:

Option A - Two reed contacts (recommended, most robust)

One contact is positioned to open when *tilting*, a second to open only on *full opening* (typically on the closing side of the sash). Both sensors are entered directly into the respective blueprint inputs – no templates needed. This is the option with the lowest error potential.

Option B - Handle sensor plus window contact

A window-handle sensor (e.g. Aqara, Bosch) provides three states: closed / tilted / open. Combined with an opening contact, both states can be derived via template helpers:

```
template:
- binary_sensor:
  - name: "Window office tilt"
    unique_id: fenster_buero_kippstellung
    device_class: window
    state: >
      {{ is_state('sensor.fenstergriff_buero', 'tilt') }}

  - name: "Window office fully open"
    unique_id: fenster_buero_ganz_geoeffnet
    device_class: window
    state: >
      {{ is_state('sensor.fenstergriff_buero', 'open') }}
```

The exact state names (**open**, **tilt**, **closed**) depend on the device and integration – please check under Developer Tools → »States«.

Option C - Window contact plus separate handle contact

If only an opening contact (**A**) and a handle contact (**B**) are available, the logic can be derived like this:

```
template:
- binary_sensor:
  - name: "Window kitchen tilt"
    unique_id: fenster_kueche_kippstellung
    device_class: window
    state: >
      {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
        and is_state('binary_sensor.griff_kueche_contact', 'on') }}

  - name: "Window kitchen fully open"
    unique_id: fenster_kueche_ganz_geoeffnet
    device_class: window
    state: >
      {{ is_state('binary_sensor.fenster_kueche_contact', 'on')
        and is_state('binary_sensor.griff_kueche_contact', 'off') }}
```

Windows without a fully-open sensor: simply leave the *window fully open* input empty. The walkthrough branch then never triggers for that shutter – everything else keeps working. Sensible for windows that are only ever tilted (guest WC, basement windows).

Pitfall: if the same contact is entered *both* as a tilt *and* as a fully-open sensor, the fully-open logic permanently blocks the ventilation logic. Only ever assign *one* of the two inputs per physical sensor.

6. Installing the blueprint

1. Place the YAML file under:
`config/blueprints/automation/<folder>/<filename>.yaml`
2. Developer Tools → **YAML** → *Reload Automations*. After changes to `configuration.yaml`, do a full restart instead.
3. Settings → Automations & Scenes → Blueprints → select the blueprint → *Create Automation*.
4. Fill in all inputs for the first shutter, save, name it.
5. Repeat for each additional shutter.

When the blueprint is updated later, it is enough to replace the file and reload the automations. The entered inputs are preserved as long as the input names stay the same. Removed inputs are ignored, new inputs fall back to their default.

7. Complete input reference

Basics

Input	Default	Meaning
<code>rollo_entity</code>	-	The shutter to control (<code>cover</code>).
<code>himmelsrichtung</code>	-	Azimuth window of the facade. Determines when »the sun is on this window«. For unusual facades, pick the option whose angle range fits best.
<code>brandalarm_entity</code>	<code>input_boolean.brandalarm_status</code>	While <code>on</code> , the blueprint performs no movements at all.
<code>hausmodus_entity</code>	-	The house-mode helper.
<code>weather_entity</code>	-	Weather entity for cross-checking.

Positions (0 % = closed, 100 % = open)

Input	Default	Meaning
<code>max_oeffnung</code>	94	Normal open position. Deliberately below 100 % if the curtain otherwise hits the mechanical stop or the position is not reported cleanly when fully up.
<code>beschattung_pos_morgens</code>	60	Shading position before the switch time.
<code>beschattung_pos_nachmittags</code>	60	Shading position after the switch time.
<code>beschattung_wechsel_zeit</code>	13:00	Switch point between the two shading positions. Guideline: the time the sun fully hits the facade.
<code>lueftungs_pos</code>	15	Minimum height with a tilted window.
<code>durchgangs_pos</code>	80	Position with a fully open window while shading or extreme-heat protection is active . Without active sun protection the shutter moves to <code>max_oeffnung</code> instead.
<code>hitze_extrem_pos</code>	10	Position under extreme heat.

Temperature thresholds

Input	Default	Meaning
<code>raumtemp_max_schwelle</code>	23.5 °C	Above this indoor temperature, heat alarm applies. With shading already active, the blueprint internally uses <i>threshold minus 1 K</i> as hysteresis.
<code>raumtemp_min_schwelle</code>	21.0 °C	Winter lock. Below this indoor temperature, shading <i>never</i> happens – the sun may heat.
<code>aussentemp_schwelle</code>	25.0 °C	Outdoor temperature above which shading is allowed. OR-linked with the indoor threshold.
<code>hitze_extrem_innen_ein</code>	25.0 °C	Indoor threshold at which extreme mode starts.
<code>hitze_extrem_innen_aus</code>	23.5 °C	Indoor threshold below which extreme mode ends.
<code>hitze_extrem_aussen_ein</code>	28.0 °C	Outdoor threshold for extreme mode.
<code>hitze_extrem_aussen_aus</code>	26.0 °C	Outdoor threshold below which extreme mode ends.

Keep a minimum gap: there should be at least 2 K between `raumtemp_min_schwelle` and `raumtemp_max_schwelle`. With too narrow a window (e.g. 21 / 22 °C) the on and off points practically coincide and the shutter flaps. Proven: **20 °C / 22.5 °C**.

Brightness and PV

Input	Default	Meaning
<code>pv_sensor</code>	empty	PV power. W or kW – detected automatically.
<code>pv_schwelle</code>	2000 W	Above this power it counts as »bright«. At twice the threshold (»blazing«) the weather check is skipped.
<code>lux_sensor</code>	empty	Illuminance sensor, ideally one per facade side.
<code>lux_schwelle</code>	25000	Above this value it counts as »bright«. The default is often too high for outdoor sensors in shade – for a

sensor on the facade, practical values are more like 9,000–15,000 lx.

If both sensors fail or are empty, the weather entity alone decides (failover).

Times and presence

Input	Default	Meaning
zeit_fruehestens	06:30	Earliest opening on workdays.
sonnenaufgang_woche	on	If active, opening on workdays also waits for sunrise.
zeit_we_fruehestens	08:00	Earliest opening on weekends, holidays and in vacation mode.
zeit_spaetestens	09:30	Hard fallback: opens at this time even without presence.
zeit_oeffnen_sperre	11:00	End of the morning phase. After this, a closed shutter is treated as <i>manually closed</i> (see chapter 9).
zeit_kontrolle	22:00	Time of the evening check for open windows.
sunset_offset	30 min	Delay after sunset before closing. Negative values = close earlier.
praesenz_sensor	empty	One or more presence sensors. If one reports on and the earliest time is reached, opening happens.
workday_sensor	empty	Holiday detection. off = late opening time.

Notifications

Input	Default	Meaning
notify_handy	empty	The service name <i>without</i> prefix. With handys the blueprint calls notify.handys .
notify_block_active	off	Enable block notifications.
pause_timer	empty	Timer that suppresses notifications while it runs.

8. Setting recommendations by orientation

Facade	Switch time	Shading morning / afternoon	Rationale
East	approx. 10:00	20-30 % / 30 %	Early, low sun blinds strongly. After noon the facade is in shade – shading ends automatically because <code>sonne_auf_f</code> becomes false.
South	approx. 11:00-13:00	50-65 % / 30-65 %	High sun. Partial shading often suffices because the roof overhang helps.
West	approx. 15:00	60 % / 40 %	Low, aggressive afternoon sun. Close further in the afternoon than in the morning – the most common misconfiguration is exactly the reverse order.
North	any	-	Shading mostly unnecessary. The blueprint is still useful for ventilation, night lowering and walkthrough.

Approach for first commissioning

1. Set up **one** shutter first and observe it for one or two sunny days.
2. Lowers too rarely? → lower the lux or PV threshold.
3. Lowers too often or under cloud? → raise the thresholds or let the weather check take effect.
4. Flapping? → increase the gap between `raumtemp_min` and `raumtemp_max` .
5. Only once one shutter runs cleanly, transfer the settings to the rest.

9. The manual override

The blueprint tries to detect whether a shutter was moved *by hand* to a position it would never approach itself. If so, it holds back the day logic and respects the user's decision.

Detection works via the current position. Considered »automatic« – with ± 2 % tolerance:

- `max_oeffnung` , both shading positions, `lueftungs_pos` , `durchgangs_pos` , `hitze_extrem_pos`
- fully closed (≤ 5 %)

Any other position counts as a manual override. Additionally: a closed shutter *after* the morning opening lock is also treated as manually closed – whoever lowers the shutters again at 11am evidently wants quiet.

Important: the override only holds back the *day logic* (shading, raising, extreme-heat protection). It is deliberately **ignored** for: fire alarm, walkthrough/cross-ventilation and evening closing. Safety and window logic always take precedence.

Side effect when configuring: if two positions happen to be very close (e.g. ventilation 30 % and shading 30 %), the blueprint can no longer tell them apart. Choose positions at least 10 % apart.

10. Interaction with a fire-alarm automation

The blueprint does **not** open on fire alarm itself – it *holds still*. Opening as an escape route is handled by a separate automation. This division of labour is deliberate: otherwise the two would overwrite each other.

Example of the corresponding automation (abbreviated):

```
alias: "Safety: FIRE ALARM"
mode: parallel
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.smoke_living_room
      - binary_sensor.smoke_hallway
      - binary_sensor.heat_kitchen
    to: "on"
actions:
  # 1. Set boolean -> the blueprint retreats immediately
  - action: input_boolean.turn_on
    target:
      entity_id: input_boolean.brandalarm_status

  # 2. Critical push notification
  - action: notify.handys
    data:
      title: "FIRE ALARM!"
      message: "Triggered by: {{ trigger.to_state.name }}"
      data:
        priority: high
        channel: alarm_stream

  # 3. Open all shutters as an escape route
  - action: cover.set_cover_position
    target:
      entity_id:
        - cover.rollo_living_room
        - cover.rollo_bedroom
    data:
      position: 100

  # 4. Escape-route lighting
  - action: light.turn_on
    target:
      entity_id: light.hallway
    data:
      brightness_pct: 100
```

The boolean is not reset by itself. While `input_boolean.brandalarm_status` is `on`, *all* shutter automations are suspended – even after a harmless false alarm from cooking fumes or dust. The automations still show `on` and look perfectly healthy; they just drop straight into branch 0 on every run. The fault is practically invisible from the outside. The two building blocks below solve this.

10.1 Acknowledged reset via push notification

An *automatic* reset would be dangerous: after a real event you do not want the shutters to close again on their own. Hence the solution with human confirmation – two additional automations.

Automation A – offer the reset

Once all detectors have been quiet for 10 minutes and the boolean is still active, a push notification arrives with action buttons:

```
alias: "Safety: FIRE ALARM offer reset"
mode: single
triggers:
  - trigger: state
    entity_id:
      - binary_sensor.smoke_living_room
      - binary_sensor.smoke_hallway
      # ... all other detectors
    to: "off"
  for:
    minutes: 10
  - trigger: state
    entity_id: input_boolean.brandalarm_status
    to: "on"
  for:
    minutes: 10
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
  - condition: state
    entity_id:
      - binary_sensor.smoke_living_room
      - binary_sensor.smoke_hallway
      # ... all other detectors
    state: "off"
actions:
  - action: notify.handys
```

```

data:
  title: "Fire alarm: detectors quiet again"
  message: >-
    All detectors have been reset for 10 minutes. The shutter control is
    still SUSPENDED. Only confirm once the situation is truly resolved.
data:
  ttl: 0
  priority: high
  actions:
    - action: BRANDALARM_RESET
      title: "Reset alarm"
    - action: BRANDALARM_IGNORE
      title: "Not yet"

```

Automation B – reset confirmed

Reacts to the button press in the notification:

```

alias: "Safety: FIRE ALARM reset confirmed"
mode: single
triggers:
  - trigger: event
    event_type: mobile_app_notification_action
    event_data:
      action: BRANDALARM_RESET
conditions:
  - condition: state
    entity_id: input_boolean.brandalarm_status
    state: "on"
actions:
  - action: input_boolean.turn_off
    target:
      entity_id: input_boolean.brandalarm_status
  - action: notify.handys
    data:
      title: "Fire alarm reset"
      message: >-
        The shutter control is active again. Normal operation within the
        next 15 minutes.

```

If the button is never pressed, nothing happens – the shutters stay up. There is deliberately **no timeout** that takes the decision out of the human's hands. After the reset the shutters do not spring into action; they take effect on the next regular run, i.e. within 15 minutes at most (`time_pattern`).

When creating these via the HA API, the validator may report that `BRANDALARM_RESET` is »not found in service registry«. That is a false alarm from the checker – it is a notification action ID, not a service name.

10.2 Dashboard card as a second safety net

In case the phone is out of reach, the boolean should also be accessible in the UI. A conditional card shows it *only* while the alarm is active and disappears by itself after the reset:

```

type: conditional
conditions:
  - condition: state
    entity: input_boolean.brandalarm_status
    state: "on"
card:
  type: tile
  entity: input_boolean.brandalarm_status
  name: FIRE ALARM – shutter control suspended
  icon: mdi:fire-alert
  color: red
  features:
    - type: toggle
grid_options:
  columns: full

```

Sensible position: at the very top of the main dashboard, directly below the person badges – there it stands out immediately in an emergency, yet does not disturb normal operation because it is invisible.

11. Troubleshooting and debugging

Reading traces

Settings → Automations → the affected automation → menu top right → **Traces**. There HA shows, for each run, which `choose` branch was chosen and with which variable values. The *Changed Variables* tab is the most informative – it lists `sonne_auf_f`, `hitze_alarm`, `beschattung_aktiv`, `is_manual_override` and `aktuelle_pos` in plain text.

Typical symptoms

Symptom	Cause and remedy
Shutter never shades	Check: indoor temperature above the winter lock? Sun within the azimuth window of the chosen orientation? Lux/PV threshold realistic? Does the weather entity report one of the recognised states?

Shutter shades constantly	Usually an outdoor threshold set too low combined with the OR logic. Raise the outdoor threshold.
Shutter flaps	Temperature thresholds too close, or two positions too close together (override detection fails to distinguish them).
Shutter does not open in the morning	No presence sensor triggered and <code>zeit_spaetestens</code> not yet reached. Or: <code>sonnenaufgang_woche</code> active and the sun is not up yet in winter.
Shutter ignores the day logic	Most likely <code>is_manual_override</code> . Move the shutter to a known position once (e.g. fully up), then the automation takes over again.
Push notifications do not arrive	The notify service does not exist. Check under Developer Tools → Actions whether <code>notify.<name></code> is listed. After creating a notify group a restart is required.
Nothing moves at all	Check <code>input_boolean.brandalarm_status</code> – if it is <code>on</code> , the blueprint is suspended. The conditional dashboard card from chapter 10.2 shows this immediately.

Testing variables live

Developer Tools → **Template**. This lets you check, for example, whether the sun is currently on the facade:

```
Azimuth:  {{ state_attr('sun.sun', 'azimuth') }}
Elevation: {{ state_attr('sun.sun', 'elevation') }}
Position:  {{ state_attr('cover.rollo_living_room', 'current_position') }}
```

12. Known limitations

- **Position-capable shutters only.** Pure open/close drives without `current_position` do not work.
- **No slat control.** Venetian blinds with a slat angle are only driven by height.
- **One shutter per automation.** This is intentional (individual parameters per window) but means correspondingly many automations with many windows.
- **No shading calculation for neighbouring buildings or trees.** The azimuth range is a rough model. If you have a row of trees in front of the west window, adjust the angle range or the lux threshold accordingly.
- **No wind protection.** For awnings or external blinds with wind risk an additional, superordinate lock would be needed.
- **The fire-alarm boolean is not reset automatically.** See chapter 10.
- The `time_pattern` every 15 minutes means: a change to the conditions takes effect after 15 minutes at worst.

13. Changelog

Version	Changes
V84	New: a fully opened window raises the shutter to <code>max_oeffnung</code> when no sun-position mode is active – also at night, before opening time and out of a manually closed state. While shading is active, still only up to <code>durchgangs_pos</code> . Fix: variable order – <code>sonne_auf_f</code> was used inside <code>hitze_extrem</code> before it was defined. The extreme-heat protection therefore did not trigger reliably. New: workday sensor as an input – holiday detection for the late opening time. New: helper variables <code>sonne_stark</code> and <code>beschattung_aktiv</code> ; disentangles the previously duplicated brightness logic.
V83	Extreme-heat protection with a separate position and hysteresis over indoor <i>and</i> outdoor temperature.
V82	Anti-flapping: PV trigger with 2 min debounce, <code>mode: restart</code> , <code>time_pattern</code> at 15 minutes.

This documentation describes the »Master Roller Shutter Control V84« blueprint. Use at your own risk – in particular, test the fire-alarm coupling once in full before productive use (set the boolean by hand and check that all shutters react as expected).